LiveRecall: Revision-Based Caption Distribution

CS3103 Assignment 4 Report
Team 23

Ngo Hoang Nhat Minh (A0301185N)
Chang Jing Jie Jordon (A0281173H)
Xinyang (A0281296W)
Jihan (A0287033E)

1 Introduction and Architecture

LiveRecall distributes a continuously revised caption stream to many concurrent clients while preserving correctness, freshness and bounded resource usage under slow readers, disconnections and constrained networks. This report describes our server design, the revision and recovery semantics it guarantees, the overload policy that keeps slow clients from degrading normal ones, and the measurements that justify both.

Our Design priority: Correctness is prioritized above freshness. Final Caption Completeness below 99% caps the performance assessment regardless of latency, and the specification treats unbounded state as a correctness failure rather than a performance one. Every bound in the system is therefore explicit and justified.

1.1 Module structure

`solution_server.py` contains only wiring for CLI parsing, logging configuration and connection dispatch. All behaviour are separated into modules, which let four people work concurrently without breaking each other's work.

Module	Responsibility	Key bound
`model.py`	Record types, revision ordering rule	—
`app.py`	Configuration, dependency container	all bounds and thresholds
`caption_state.py`	Per-segment state, final durability	512 segments
`history.py`	Recovery window, HISTORY replies	90 s / 1500 items
`hub.py`	Client registry, fan-out	—
`client_session.py`	Per-client queue, writer, overload policy	32 soft / 128 hard frames; 3 in flight
`ingest.py`	Parsing, validation, source handler	—
`metrics.py`	Counters and observability	—

Table 1: Server modules. `model.py` and `app.py` are shared interfaces; every other module has exactly one owner.

1.2 Data flow for one caption revision

A caption revision follows this path:

1. **Validate** (`ingest`): incoming captions are ingested and invalidated. All untrusted input stops at this boundary.
2. **Admit** (`caption_state.apply`): returns the update only if it advances state, and `None` for anything stale or duplicate. Rejected revisions therefore cannot reach the recovery window or any client queue.
3. **Record** (`history.record_final`): a final caption enters the recovery window *before* publication, so a client reconnecting within the same event-loop can read the caption history.
4. **Publish** (`hub.publish`): send the caption frame to every client queue synchronously, never awaiting a socket.

Step 4 is why one unresponsive client cannot stall the broadcast path. The naive baseline awaited every `send()` on the source's own task; here the source only ever appends to bounded queues, and a dedicated writer task per client drains to its own socket.

2 Protocol and Revision Semantics

We implement the mandatory compatibility interface without extensions, which eliminates evaluator-compatibility risk. Messages are SOURCE_HELLO, CAPTION_UPDATE, CLIENT_HELLO and HISTORY.

2.1 Three invariants

Revision handling must maintainthree rules (invariant) in this priority order:

1. **Final durability.** Once a segment is final, nothing replaces it, even if it is a higher revision or a retransmission. This check precedes the revision comparison, so revision 99 still loses to a final revision 3.
2. **Monotonic revisions.** Within a segment a higher revision replaces a lower one; a lower one is never applied.
3. **Duplicate suppression.** A revision already held is rejected and not republished.

Because all three are enforced at one admission point that returns `None` on rejection, no downstream component re-implements ordering.

3 Recovery and Bounded State

3.1 Resume semantics

`last_segment`/`last_revision` are interpreted as a **watermark** which is the latest *finalised* state the client already holds. HISTORY returns only finals strictly newer than that watermark, and no older than the requested window. If a client holds segment 12 but missed segment 10, its watermark hides the gap and no bounded history can repair it. The overload policy must never discard a final, and the provisional revisions are the only droppable traffic.

3.2 Why the window is bounded twice

The recovery window is bounded by age (90 s) *and* by item count (1500). The count bound is because a bursty source can place far more than the expected number of segments inside a 90-second window, so a purely time-based bound would still grow with the source rate. Live per-segment state is separately capped at 512 tracked segments, with an eviction watermark that keeps evicted segments rejected so eviction cannot resurrect stale revisions, plus a hard backstop for a source that abandons a segment without finalising it.

3.3 Measured recovery behaviour

Against a running server, with 16 segments finalised:

Client state	Items returned	Interpretation
$(0, 0)$, 30 s	16, segments 1–16	Late join receives the whole window
$(5, 3)$, 30 s	11, segments 6–16	Watermark correctly excludes 1–5
$(0, 0)$, 5 s	7, segments 10–16	Time cutoff is applied

Table 2: HISTORY contents for different resume states.

4 Backpressure and Overload Policy

4.1 A negative result that shaped the design

We first triggered the overload policy on application queue depth, but with 10 % slow readers lagging by about 13 s, peak queue depth was only **1**. Probing a 500 ms reader showed why: its first 38 frames were

acknowledged by pongs within 2 ms while it was already falling behind, and the first late pong (frame 40) took 5.8 s. The client's WebSocket library buffers 16 frames before pausing, and kernel and transport buffers absorb more, so *queue depth is not a usable lag signal.*

4.2 The policy we adopted

Lag signal. Each session pings its own connection. A ping shares the caption byte stream, so its pong returns only after the client has read everything sent before it; RTT above the connection's minimum is the queueing delay in front of that client, and added network delay alone does not count. Browsers and `websockets` answer pings automatically, so the protocol is unchanged.

Two modes. In *normal* mode every revision is sent in order. A client becomes *overloaded* when delay exceeds `overload_lag_high` or its queue exceeds the soft bound, and clears only after delay stays below `overload_lag_low`, with a short queue and no coalescing, for `overload_clear_hold` seconds. While overloaded:

- **Coalescing.** A newer revision overwrites a queued older provisional of the same segment in place; a final overwrites its own pending provisional. The queue holds at most one frame per active segment plus pending finals. Finals are never dropped; a client whose queue still exceeds the hard bound after coalescing is closed with code 1013 and resumes through HISTORY.
- **In-flight window.** At most `overload_inflight_frames` unconfirmed frames, so the backlog waits in our queue, where it can be coalesced, not in kernel buffers.
- **Rate pacing.** The client's read rate is estimated from frames confirmed between two late pongs, when its queue is at the same level (within about 6 % of the true 2 messages/s). Sends are paced at 0.5× that rate after a late pong, then 0.85×, rising 10 % every 5 s without one, so a recovered client returns to full updates.

Parameter	Value	Reason
`overload_lag_high`	0.25 s	half the 500 ms freshness threshold
`overload_lag_low`	0.08 s	hysteresis gap
`overload_clear_hold`	2 s	four probe intervals
`overload_inflight_frames`	3	keeps backlog in our queue
`probe_interval`	0.5 s	≤2 pings/s per healthy client

Table 3: Overload-policy parameters.

4.3 Why this is a correctness mechanism

In T3 a 500 ms reader consumes about 118 of the 228 revisions emitted in 60 s. Spending that budget on finals and newest state rather than arrival order raises Final Caption Completeness from 95.13 % to 99.34 %, across the 99 % gate (§5.5).

5 Experiments

All runs use the supplied harness with unmodified measurement logic, on Ubuntu Linux (kernel 6.6.87.2, 4 vCPUs, 7.8 GiB RAM, Python 3.13.5). Each reported figure is the median of three independent runs; `run_demo.py --runs 3` restarts the server for every run.

5.1 Results

Duplicate and stale caption counts were **zero** across every run, as were invalid non-final HISTORY items.

Test	P50 (ms)	P95 (ms)	P99 (ms)	Fresh (%)	Complete (%)	Coverage (%)
T1 20 clients, 20 s	4.17	12.13	14.99	100.00	100.00	100.00
T2 100 clients, 40 s	11.38	29.39	36.66	100.00	100.00	100.00
T3 100 clients, 10 % slow	14.48	2265.89	8414.06	94.65	99.34	95.18
T4 100 clients, 20 % recon.	10.58	25.78	30.95	100.00	100.00	98.33
T5 delay+jitter+3 % loss	375.82	1928.09	6750.50	67.29	97.06	58.63

Table 4: Public test results. Values are medians of three independent runs. P50/P95/P99 are caption age in milliseconds; Fresh is the fraction at or below 500 ms. T5 applies 120 ms mean delay, 30 ms jitter and 3 % loss on the loopback interface.

Population	P50 (ms)	P95 (ms)	Fresh (%)
Normal clients (90 %)	13.88	28.87	100.00
Slow clients (10 %)	5550.85	9357.85	1.69

Table 5: Normal clients are unaffected by deliberately slow readers.

5.2 Slow-reader isolation (T3)

The server log shows the policy engaging exactly where intended: `overload_engaged`=10, matching the 10 slow readers, with no normal client affected; `provisionals_coalesced`=1020; and peak queue depth 9, against a soft bound of 32.

5.3 Reconnect and recovery (T4)

With 20 % of clients disconnected for 5 s, final completeness remained 100.00 % with zero duplicate or stale captions, and the HISTORY response arrived with a median of 4.37 ms and P95 of 7.97 ms. This is measured from the completed WebSocket handshake to HISTORY receipt, so it covers the CLIENT_HELLO round trip plus history construction; the bounded window and indexed lookup keep this recovery path within a single-digit millisecond tail in this experiment.

5.4 Network impairment (T5)

With `delay_loss.sh lo` active, the three-run medians were 375.82 ms P50, 1928.09 ms P95 and 6750.50 ms P99. Fresh Caption Ratio fell to 67.29 %, while final completeness remained 97.06 %; duplicate, stale and connection-error counts remained zero. WebSocket runs over TCP, so packet loss does not normally lose application messages: retransmission and head-of-line blocking instead inflate caption age, especially in the tail. The completeness reduction reflects captions that remained undelivered when the fixed experiment ended, rather than duplicate or stale state.

5.5 Before/after comparison

We compare the final design against the naive baseline (commit `4c7e5c9`: a single shared broadcast via `asyncio.gather`, no recovery history, no per-client queues), using the same unchanged harness and T3 parameters. Each figure is the median of three runs.

Trade-off. The overload policy raises completeness and cuts slow-client lag by roughly two thirds, but suppressing superseded provisionals reduces coverage for the affected clients. Coverage returns to 100 % once no client is lagging: T2 reports 100.00 % in all three runs, and in T3 the 90 normal clients received all 20 520 revisions. This is why the policy is triggered by measured lag rather than applied unconditionally. The cost is a 10.09 ms rise in normal-client P95, most likely from per-client writer tasks and ping probes.

Metric (T3 parameters)	Baseline	Final	Change
Final completeness (%)	95.13	99.34	+4.21 pp
Slow-client P50 / P95 age (s)	14.77 / 29.18	5.55 / 9.36	−62.4 % / −67.9 %
Overall P99 age (ms)	25120.57	8414.06	−66.5 %
Normal-client P95 (ms)	18.78	28.87	+10.09 ms
Slow-client freshness (%)	1.69	1.69	unchanged
Normal update coverage (%)	95.18	95.18	unchanged

Table 6: Before/after under identical parameters.

6 Failures, Limitations and Future Work

6.1 Failures

- **Queue depth was the wrong lag signal** (§4.1): the backlog sits in client-library and kernel buffers, invisible to the server, so we switched to ping RTT above the minimum.
- **Arrival-order delivery wastes slow readers' budget** on obsolete provisionals; coalescing fixed it (§4.3).

6.2 Limitations

- A client offline for more than 90 s has missed finals that no bounded history can return. This is the direct cost of refusing to retain an unbounded transcript.
- Slow readers still lag by about 5 s and miss the last few finals of a run, which are still in transit when the test ends. The client library buffers 16 frames before any server-side signal can see lag, so detection always costs about 16×500 ms; removing it needs application-level acknowledgements, which the fixed protocol lacks.
- One active session, as permitted by the specification. Multi-session routing is not implemented.
- Currently, network impairment increases tail age. Under T5, P99 age rose to 6.75 s and fresh-caption ratio fell to 67.29 %, while duplicate and stale counts stayed at zero. This is the expected reliable-TCP trade-off under loss and head-of-line blocking.

6.3 Future work

The main remaining cost is detection delay: the server only sees a slow reader once its 16-frame library queue is full. An optional protocol extension for client credits or acknowledgements would let the server observe consumption directly instead of inferring it from late pongs, removing that delay and improving slow-client freshness.

A second improvement would be adaptive history sizing based on observed reconnect gaps, subject to a hard memory cap, rather than a fixed 90-second/1,500-item history.

Appendix A: Reproduction

```
python -m venv venv && ./venv/bin/pip install -r requirements.txt
python -m unittest discover -s tests -t .
ss -ltnp | grep 8765          # must be empty before a measured run
python run_demo.py --clients 100 --duration 60 --runs 3 \
    --slow-percent 10 --slow-ms 500 --output-stem t3
```

Reported commands:

```
python -m unittest discover -s tests -t .
python run_demo.py --clients 20 --duration 20 --runs 3 --output-stem t1
python run_demo.py --clients 100 --duration 40 --runs 3 --output-stem t2
python run_demo.py --clients 100 --duration 60 --runs 3 \
    --slow-percent 10 --slow-ms 500 --output-stem t3
python run_demo.py --clients 100 --duration 60 --runs 3 \
    --disconnect-percent 20 --disconnect-at 15 --disconnect-seconds 5 \
    --output-stem t4
sudo ./network_profiles/delay_loss.sh lo
python run_demo.py --clients 100 --duration 40 --runs 3 --output-stem t5
sudo ./network_profiles/reset.sh lo
```